Multi-drone patrolling: persistent coverage with heuristic search

Emile Descroix - Anaël Ernadote

Modeling and Problem Solving - M2
Université Paris-Dauphine - PSL

Abstract

How can N drones be coordinated to monitor an area continuously, so that no place stays "blind" for too long? The report follows this approach: we implement an algorithm, measure its performance, identify its limits, then move on to another algorithm from the course that may fix them. We thus go through two families of heuristic search methods: first *reactive / MAPF* strategies (Dijkstra, space-time A^*, ALT, Conflict-Based Search, Partial Move A^*), then a *global optimization* of patrol cycles (hill climbing, genetic algorithm, NRPA). The main result is that bounding the idleness with optimized cycles divides the worst idleness by a factor of ≈ 2.5, whereas refining the reactive coordination alone does not.

Contents

1 Introduction **3**

2 Modeling **3**

3 Approach: two families of algorithms **4**

4 Family A: reactive strategies and MAPF **4**

4.1 Greedy baseline (Dijkstra) 5

4.2 Space-time A* 5

4.3 ALT heuristic 5

4.4 Conflict-Based Search (CBS) 6

4.5 Partial Move A* 6

4.6 Summary of family A 7

5 Family B: global optimization with cycles **8**

5.1 Reduction to a multi-TSP 8

5.2 Hill climbing 8

5.3 Genetic algorithm 9

5.4 NRPA 9

6 Consolidated results and discussion **10**

7 Conclusion **11**

1 Introduction

Monitoring an area with a fleet of drones is not a classic *coverage* problem: visiting each cell once is not enough, each cell must be *revisited regularly*. This variant is called *multi-agent patrolling*. It arises whenever a phenomenon evolves over time (emerging fires, intrusions, environmental measurements) and information gets older as long as it is not observed.

The problem combines several building blocks from the heuristic search course: shortest paths (Dijkstra, A*), *Multi-Agent Path Finding* (MAPF) and its solvers (prioritized planning, Conflict-Based Search, Partial Move A*), and combinatorial optimization with metaheuristics (local search, genetic algorithm, NRPA).

The report is deliberately **sequential**: each section implements an algorithm, evaluates its performance on the same reference instance, then motivates the next algorithm by the limit observed. All figures are produced by the project code (`generate_figures.py`) to ensure reproducibility.

2 Modeling

Environment. The area is a $W \times H$ grid with 4-connectivity. Some cells are *obstacles* (no-fly zones that cannot be crossed). A drone moves by one cell per time step (North, East, South, West, or wait), exactly under the MAPF assumptions of the course: *"each agent can move N, E, S or W into any adjacent unblocked cell"*.

Sensor. A drone on cell c observes every free cell within a Euclidean disk of radius R around c. The view is not occluded: drones fly over obstacles.

Idleness. Each free cell c has an *idleness* $\tau(c,t)$: the number of steps since it was last observed. At each time step, every cell ages by 1, and observed cells go back to 0:

$$\tau(c,t{+}1) = \begin{cases} 0 & \text{if } c \text{ is observed at } t{+}1, \\ \tau(c,t)+1 & \text{otherwise.} \end{cases}$$

Objective. We want to minimize the **maximum idleness** in steady state, $\max_c \tau(c,t)$ (the worst blind area), and secondarily the mean idleness. Figure 1 illustrates the stakes: a greedy strategy can leave a large area unmonitored (light on the heat map), whereas a cycle-based strategy keeps a uniform coverage.

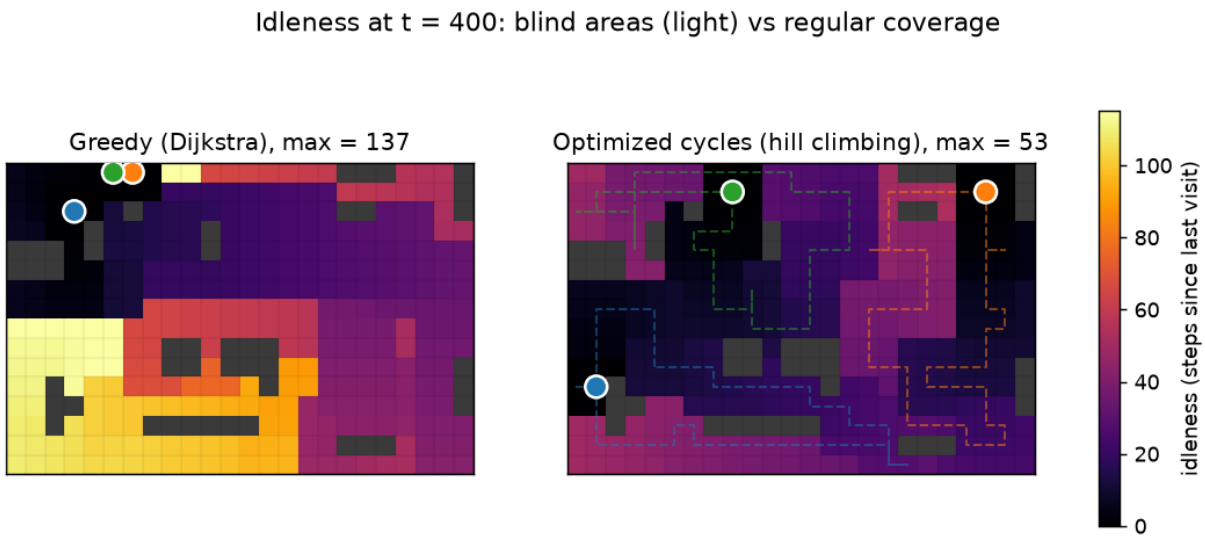


Figure 1: Idleness map at $t = 400$ (24×16 grid, 3 drones). Left, the greedy baseline: the two top drones have clustered together and a whole region (light yellow) has stayed blind for ≈ 137 steps. Right, the optimized cycles (dashed lines): coverage is regular and the maximum idleness drops to 53.

Reference instance. Unless stated otherwise, all evaluations use the same instance: 24×16 grid, 12 % obstacles, radius $R = 1.5$, 3 drones, random seed 42, and 400 simulation steps, of which only the second half is averaged (steady state).

3 Approach: two families of algorithms

We compare two paradigms, both taken from the course.

Family A: reactive / MAPF. At each time step, each drone picks a "cold" target and a solver coordinates the moves without collision. We progressively refine the *pathfinding* (Dijkstra, then A*, then ALT) and the *coordination* (priorities, then CBS, then PMA*).

Family B: global optimization. We precompute patrol *cycles* that *bound* the idleness, then optimize these cycles with metaheuristics (hill climbing, genetic algorithm, NRPA).

The common thread of the following sections: family A improves *safety* and smoothness, but runs into a variable and rather greedy coverage; family B changes paradigm and brings the idleness down.

4 Family A: reactive strategies and MAPF

All these strategies share the same *target choice*. A drone at position p targets the cell c that maximizes a score combining urgency and proximity:

$$\text{score}(c) = \tau(c, t) - \alpha\, d(p, c), \tag{1}$$

where $d(p, c)$ is the shortest-path distance and α a weight. Only the way paths are *planned* and *coordinated* changes.

4.1 Greedy baseline (Dijkstra)

Implementation. The distance d in equation (1) is computed with **Dijkstra**, then the drone follows the shortest path to its target. There is no coordination: we only prevent two drones from picking the same target, and move conflicts are resolved naively by index priority (the lowest index moves, the other waits).

Evaluation. It is a fair baseline: maximum idleness ≈ 136, mean ≈ 52 (table 1). But the policy is optimal at time t and not at $t + 1$: two drones can converge to the same region and abandon another one (figure 1, left). Moreover, resolving conflicts by simple priority can make two drones overlap in edge cases.

Limit ⇒ next step. We need a coordination that *guarantees* the absence of collisions: we move on to space-time planning.

4.2 Space-time A*

Implementation. We plan in the state space (cell, t). A* expands the leaf with the smallest $f = g + h$, using the Manhattan heuristic $h(c, g) = |x_c - x_g| + |y_c - y_g|$, which is admissible because it ignores obstacles and other drones. The paths of higher-priority drones are written into a **reservation table** that forbids vertex collisions ((c, t) occupied) and edge collisions (head-on swaps). The table is never built explicitly: reserved states are simply not *generated*. This is *prioritized planning* / *Cooperative Pathfinding* by Silver (2005).

Evaluation. Safety is achieved (zero collisions), but the idleness does not improve; it even gets worse (≈ 161, table 1) because the waits imposed by priorities slow down the coverage. Above all, the priority order is *fixed* (drone index): the method is **neither complete nor optimal**.

Limits ⇒ two directions. (i) The Manhattan heuristic is poorly informed in corridors: we improve it with ALT (§4.3). (ii) Priority-based coordination is fragile: we replace it with CBS (§4.4) then PMA* (§4.5).

4.3 ALT heuristic

Implementation. **ALT** (A*, Landmarks, Triangle inequality) precomputes, with a Dijkstra from a few *landmarks* L, the true distances $d(L, \cdot)$. The triangle inequality then gives an admissible lower bound that is tighter than Manhattan:

$$h_{\text{ALT}}(c, g) = \max_{L} \big| d(L, c) - d(L, g) \big| \;\leq\; d(c, g). \tag{2}$$

Landmarks are chosen "farthest-first" (as far apart as possible). Unlike Manhattan, ALT "sees" the obstacles.

Evaluation. ALT does not change the patrol quality (same optimal paths): it is a *speed-up*. On a corridor grid (40×28, 30 % obstacles), A* expands $\approx 1.4\times$ fewer leaves (figure 2), while optimality is preserved. The gain grows with the obstacle density.

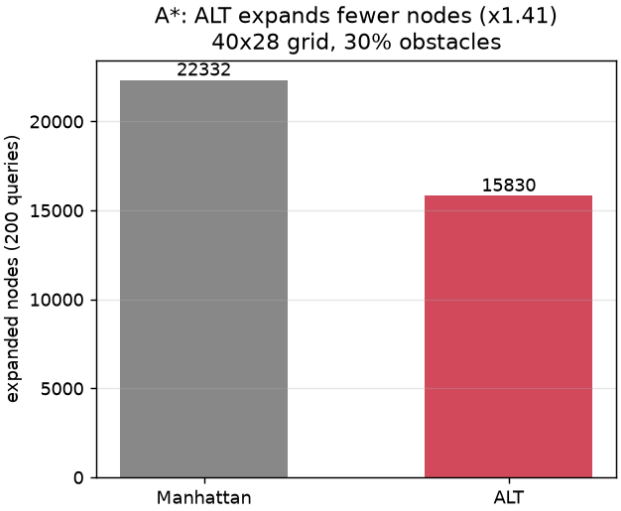


Figure 2: ALT versus Manhattan: total number of leaves expanded by A* over 200 shortest-path queries. The paths found are identical; ALT simply explores fewer nodes.

4.4 Conflict-Based Search (CBS)

Implementation. **CBS** fixes the weakness of prioritized planning. It works on two levels:

- *Low level*: for a given drone, a space-time A* finds the shortest path under a set of *constraints* (vertex or edge bans at given times).
- *High level*: a binary tree. Each drone is first planned independently; we look for the first conflict; we create two children, each forbidding *one* of the two conflicting drones from being there at that time, and we replan only that drone. The node with the lowest cost (sum of lengths) is always expanded first.

Unlike space-time A*, CBS fixes no priority order: it is **complete and optimal**.

Evaluation. CBS does produce collision-free trajectories. It *smooths* the coverage: it is the reactive strategy with the lowest *mean* idleness (≈ 49, table 1). However, the *maximum* idleness stays at ≈ 138, in the same range as the baseline: better coordination does not fix the intrinsically greedy nature of equation (1).

4.5 Partial Move A*

Implementation. We can search directly in the *joint* space of the N drones (their simultaneous positions). This is optimal but explodes: a full move is a combination of one action per drone, i.e. up to 5^N successors per node. **Partial Move A*** keeps the same space but *stops the search after each partial action*: a node fixes the cells of drones $0..k-1$, and only one drone is added at a time, immediately pruning collisions with the drones already fixed. The branching factor drops from 5^N to 5.

Evaluation (as a solver). On position-swap instances, the gain grows strongly with the number of drones: PMA* generates up to $\approx 18\times$ fewer successors than joint A* for 4 drones (figure 3), for the same optimal path.

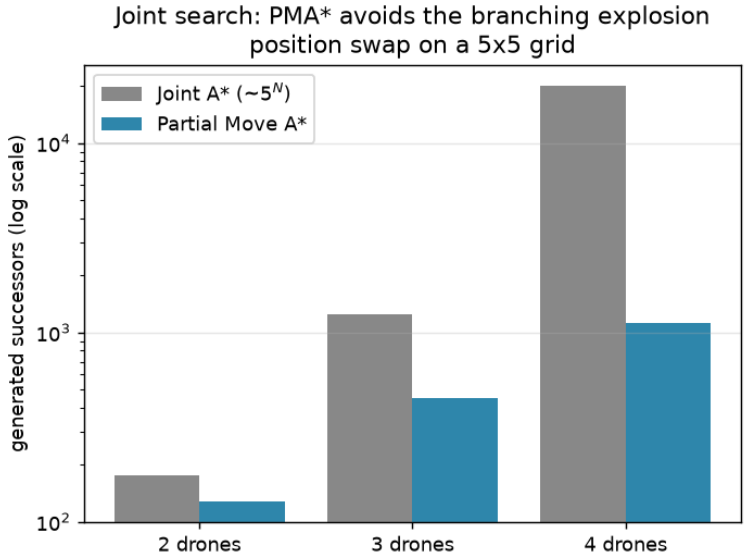


Figure 3: Joint search: generated successors (log scale). Joint A* suffers from a 5^N branching factor; PMA* avoids it with partial expansion. The gap widens as the number of drones grows.

Evaluation (in patrolling). Once integrated into the patrol, PMA* is nevertheless the *worst* ($\approx$ 191, table 1). The reason is instructive: its joint plan is optimal in *makespan*, so drones that reach their target early *wait* for the others before everyone is replanned. *This forced waiting degrades the coverage.* This is a central lesson: **MAPF optimality is not the patrol objective**. The real value of CBS and PMA* is as *solvers*, not as coverage policies.

4.6 Summary of family A

Figure 4 overlays the idleness over time for all strategies. The reactive curves (greedy, A*, CBS, PMA*) *drift*: the worst area keeps getting darker, because the target choice remains local. None of these methods *guarantees* that a given cell will be revisited. This is precisely what family B provides.

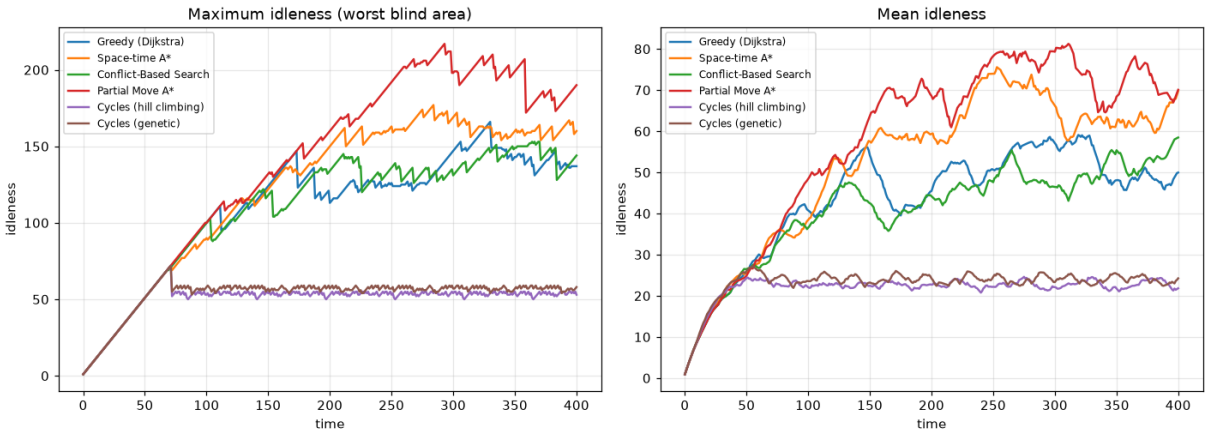


Figure 4: Maximum (left) and mean (right) idleness over time. Reactive strategies (family A) drift upwards; cycle-based strategies (family B, purple/brown) settle low and flat.

5 Family B: global optimization with cycles

5.1 Reduction to a multi-TSP

The key idea: if a drone follows a closed *cycle* of L steps, each cell of its area is revisited at least once every L steps. The maximum idleness is then **bounded** by the length of the longest cycle, which reactive strategies never guarantee. The cycles are built in three steps:

1. **Coverage**: a greedy *set cover* heuristic picks a small set of *waypoints* such that every free cell is seen from at least one of them.

2. **Distances**: one Dijkstra per waypoint gives the matrix D of shortest paths between waypoints.

3. **Tours**: the waypoints are partitioned among the N drones and each part is ordered into a cycle, which is a *multi-TSP*. The cost to minimize directly approximates the patrol objective:

$$\text{cost}(\text{routes}) = \max_d L_d + 0.1 \cdot \overline{L}, \tag{3}$$

   where L_d is the cycle length of drone d and $\overline{L}$ the mean (the mean term breaks ties between solutions with the same longest cycle).

This multi-TSP is optimized with three metaheuristics, compared **with the same evaluation budget**.

5.2 Hill climbing

Implementation. Local search on the routes, with three classic tour operators: *2-opt* (reverse a segment), *relocate* (move a waypoint to the best insertion point of another cycle), *swap* (exchange two waypoints between cycles). Sideways moves (plateaus) are accepted and the search restarts randomly on stagnation, keeping the best solution found. Moves target the longest cycle first, since it sets the cost (3).

Evaluation. This is the big leap: maximum idleness ≈ 54, i.e. divided by ≈ 2.5 compared with family A (table 1). Figure 5 shows the cycles obtained: three compact and disjoint loops, each assigned to one drone.

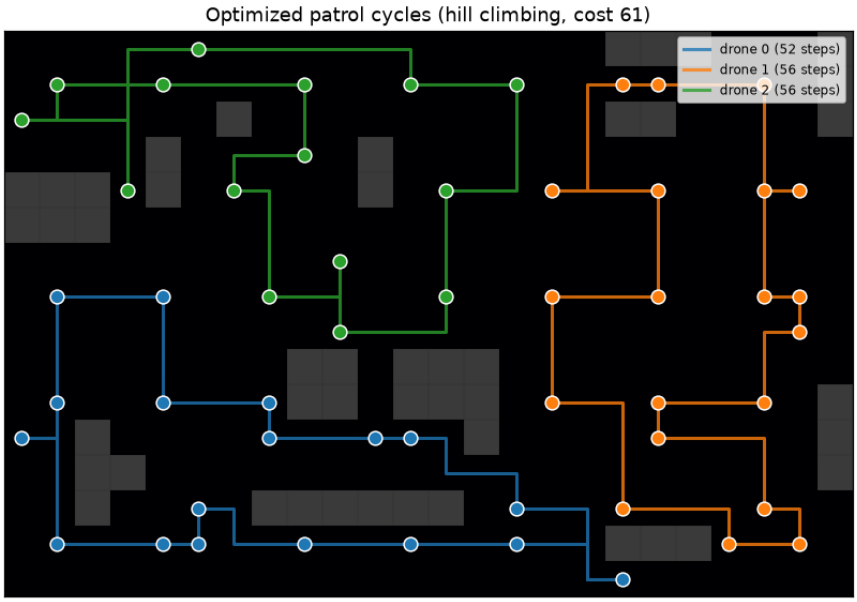


Figure 5: Patrol cycles optimized by hill climbing. Each drone loops over a compact region; the length of the longest cycle bounds the maximum idleness.

5.3 Genetic algorithm

Implementation. Multi-TSP representation:

$$\text{chromosome} = (\text{permutation of the waypoints},\ N-1 \text{ cuts}).$$

OX crossover (*Order Crossover*), mutations (swap, segment reversal, cut shift), tournament selection and elitism.

Evaluation. With the same budget, the genetic algorithm is slightly behind hill climbing (≈ 57 versus ≈ 54). On a TSP-like landscape, 2-opt local search exploits the neighborhood better than a GA without built-in local search.

5.4 NRPA

Implementation. **NRPA** (Nested Rollout Policy Adaptation; Dang *et al.* 2023) explores neither a neighborhood nor a population: it *learns a construction policy*. We use the *route-first, split-second* formulation, standard and efficient for the multi-TSP:

1. NRPA learns a single large tour through all the waypoints: a classic TSP, a setting where NRPA excels. A "move" picks the next waypoint, encoded by the edge (last, next), with a probability given by the policy.

2. This tour is split into N contiguous segments (one per drone) by dynamic programming that minimizes the longest cycle.

The `adapt` function reinforces the edges of the best tour; the recursion nests several levels.

Evaluation. With the same budget (6000 evaluations), NRPA converges more slowly than hill climbing (cost $\approx$ 118 versus 61), because local search dominates on a TSP of this size. Its correctness is confirmed by the convergence with a growing budget (cost $\approx$ 88 at 30 000 evaluations): NRPA catches up with large budgets, where its world records on harder combinatorial problems lie. Figure 6 overlays the three convergence curves.

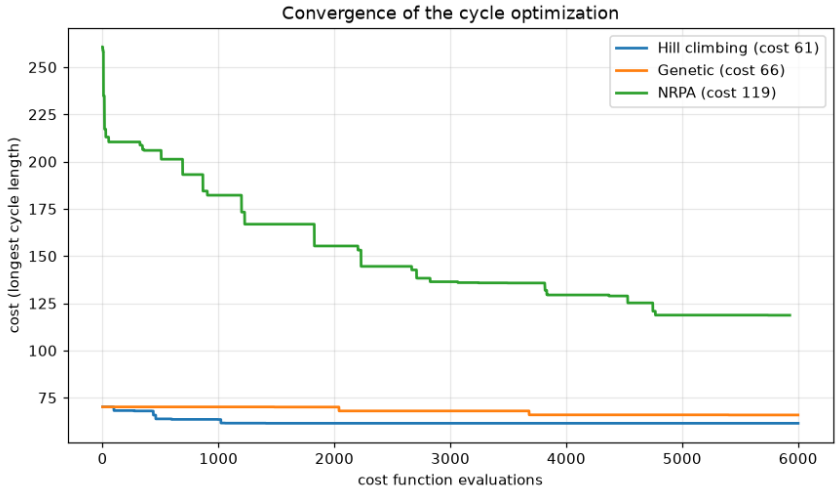


Figure 6: Convergence of the three metaheuristics on the multi-TSP, with the same evaluation budget. Hill climbing goes down the fastest; NRPA converges more slowly at this budget.

6 Consolidated results and discussion

Table 1: Steady-state idleness (24×16 grid, 12 % obstacles, 3 drones, seed 42, 400 steps). Lower is better.

Family	Strategy	Max idleness	Mean idleness
A: reactive	Greedy (Dijkstra)	135.8	52.1
A: reactive	Space-time A*	161.4	65.3
A: reactive	Conflict-Based Search	138.4	49.2
A: reactive	Partial Move A*	190.7	73.3
B: cycles	**Cycles (hill climbing)**	**53.7**	**22.7**
B: cycles	Cycles (genetic)	57.2	23.9

Three lessons emerge.

1. Global optimization changes the order of magnitude. Moving from a reactive policy to optimized cycles divides the worst idleness by $\approx$ 2.5. This is the direct consequence of the

bound guaranteed by the cycles: the worst case is *under control*, whereas reactive strategies offer no revisit guarantee (figure 4).

2. Within the reactive family, coordination matters little for the maximum idleness. CBS and the greedy baseline are neck and neck on the *max* (≈ 138 versus 136); CBS only improves the *mean*. Improving the MAPF solver does not fix the underlying flaw, which justifies the change of paradigm.

3. MAPF optimality $\neq$ good patrolling. PMA*, although an excellent joint search *solver* (figure 3), is the *worst* at patrolling: its makespan synchronization makes the drones wait. The right criterion is not the makespan of an isolated move, but the idleness *integrated over time*.

Algorithmic cost. Reactive strategies replan at every step but remain cheap (small grid); CBS and PMA* make coordination heavier. On the optimization side, hill climbing and the genetic algorithm converge in a fraction of a second for 6000 evaluations; NRPA is more expensive (a few seconds) because of the dynamic programming split repeated at each *rollout*.

7 Conclusion

Following an incremental approach (implement, measure, fix), we went through a large part of the heuristic search algorithms of the course, applied to the same concrete problem. **The right algorithm depends on the right criterion**: optimal MAPF solvers (CBS, PMA*) shine at collision-free motion but do not optimize persistent coverage, whereas the multi-TSP reduction directly aligns the optimized cost with the maximum idleness and brings it down by a factor of 2.5.

Future work. Running the family B cycles with CBS coordination would guarantee the absence of collisions *without* losing the idleness bound; a memetic GA or NRPA with a larger budget would tighten the optimization; finally, idleness weighted by priority areas, heterogeneous drones or failures would bring the model closer to a real deployment.